

PRODUCT REQUIREMENTS DOCUMENT

Quicky

“You don’t just order fast — you order smart.”

Hyperlocal quick-commerce with expiry, freshness & warranty transparency

Document status	Draft v1.0 — for review
Date	29 June 2026
Author	Product (working draft)
Scope	MVP — single-city pilot

Table of Contents

1. Overview

1.1 Summary

Quicky is a hyperlocal quick-commerce platform that delivers from nearby kirana and local stores, differentiated not by delivery speed but by giving customers visibility and control over product freshness, expiry, and warranty — and giving stores a structured way to sell near-expiry inventory before it becomes a write-off.

This document defines the problem, target users, MVP scope, detailed feature requirements, data model, success metrics, and rollout plan for a single-city pilot. It is written to be buildable: every feature below includes what the user sees, what the store/ops side has to do to support it, and the open question that still needs an answer.

1.2 Why this document exists

The original concept (expiry-based filtering, freshness meter, dynamic pricing, warranty transparency) is a strong wedge, but three things have to be resolved before a single line of code is written, because they change the shape of the product:

- **Who tags expiry data, and how.** Most kirana stores have no digital inventory. If tagging is manual and store-owner-driven, the MVP has to be designed around low-effort, low-tech data entry — not assume an API feed that doesn't exist yet.
- **Who sets the discount on near-expiry stock.** Pricing authority (platform vs. store owner) determines whether “freshness pricing” is trustworthy or arbitrary. This needs a clear rule before the UI can promise discounts.
- **Whether warranty tracking belongs in v1 at all.** Quick-commerce delivers very little electronics volume, and warranty data is usually already on-pack. This PRD treats warranty as a v2 feature and keeps MVP scope to grocery/FMCG expiry, where the customer pain is real and provable.

Each section below states assumptions explicitly so they can be challenged before build, not after. Three specific open items from the first draft of this PRD now have working answers in this version: the Use Today discount band by category (§ 8.2), the tagging-flow usability test protocol (§ 9.3), and Delhi-specific basket size / delivery cost estimates (§ 8.1) — all still marked as estimates to validate with real pilot data, not final numbers.

2. Problem Statement

2.1 For customers

- Customers have no visibility into how fresh a grocery item is at the point of online ordering — existing quick-commerce apps (Blinkit, Zepto, Instamart) show a product photo and a price, never a manufacturing or expiry date.
- Budget-conscious households want the option to buy near-expiry stock at a discount but have no legitimate way to find it; today this only happens informally, in person, at physical stores.
- There's no trust signal that an online grocery order won't arrive with a short remaining shelf life — a common frustration with current platforms.

2.2 For local stores

- Near-expiry inventory is typically marked down in-store (if at all) and often becomes a write-off, directly hitting thin kirana margins.
- Local stores have no digital channel to compete with quick-commerce platforms on anything other than price, and big platforms' dark-store model structurally excludes them.

2.3 Why now

Indian quick-commerce has trained urban consumers to expect 10–30 minute delivery, but the category is purely speed-optimized and commodity-priced — it has not touched transparency or waste. That gap is open precisely because it's not the thing incumbents are optimizing for; entering on speed alone against Blinkit/Zepto/Instamart's scale and capital is not a fight a new entrant can win.

3. Goals & Non-Goals

3.1 Goals (MVP / pilot)

1. Prove that customers will choose a discounted, near-expiry product over a full-price alternative when both are clearly labelled, in a single pilot neighbourhood.
2. Prove that participating stores see a measurable reduction in near-expiry write-offs within the pilot period.
3. Establish a manual-but-workable expiry-tagging workflow that a non-technical store owner can sustain without dedicated staff.
4. Validate delivery unit economics (cost per order vs. commission + delivery fee) at small basket sizes before scaling spend.

3.2 Non-goals (explicitly out of scope for MVP)

- Warranty tracking / electronics category — deferred to v2 (see § 9).
- Real-time inventory sync via store POS integration — deferred until manual tagging is proven and store density justifies the integration cost.
- Sub-15-minute delivery as a headline promise — the MVP targets a realistic 20–40 minute window; speed is not the differentiator and overpromising here creates support load without product benefit.
- Multi-city expansion, franchise/dark-store ownership, or B2B analytics products — all deferred until pilot signal is positive.

4. Target Users & Personas

4.1 Customer personas

Persona A — The Budget Optimiser

- Hostel/PG student or young working professional, price-sensitive, orders groceries 3–5x/week in small baskets.
- Will actively choose a 30%-off, use-today loaf of bread over full-price “fresh” stock. This is the primary early adopter and the clearest product-market fit signal to chase first.

Persona B — The Freshness-First Family Buyer

- Manages household groceries, wants to avoid being sent the most expired item in stock by an algorithm optimising for store-side clearance.
- Wants the opposite filter behaviour from Persona A: maximum remaining shelf life, willing to pay slightly more for it. The product must serve both without making the discount-seeking flow feel like a penalty for the freshness-seeking one.

Note on “health-conscious buyers” as a segment

The original concept listed health-conscious buyers as a target segment. On reflection, this group is closer to Persona B (freshness-first) than to the waste-reduction story, and shouldn't be marketed to with discount-led messaging. Treat Persona A and B as the two real segments; collapse the rest into them.

4.2 Store partner persona

The Margin-Squeezed Kirana Owner

- Runs a single store, no POS system or only a basic billing app, manages inventory by memory and eyeballing shelves.
- Primary motivation is recovering money currently lost to unsold near-expiry stock — not technology adoption for its own sake. Onboarding has to assume near-zero comfort with apps beyond WhatsApp-level usage.

5. User Flow (MVP)

The flow below is the complete MVP path — deliberately simple, with no warranty step and no AI recommendation engine in v1.

1. Customer opens the app and selects a category (MVP: grocery & daily essentials only).
2. Customer applies a freshness filter: “Use Today” (discounted), “Fresh Stock” (standard/premium price), or “Any” (default, no filter).
3. App shows matching products from stores within delivery radius, each card showing price, discount badge (if any), and a Freshness Meter.
4. Customer adds items to cart; cart clearly separates “Use Today” items from “Fresh Stock” items so the discount logic is transparent at checkout.
5. Customer places order → order routed to the nearest matching store → delivery partner assigned → delivery in a realistic 20–40 minute window.
6. Post-delivery: customer can rate freshness accuracy (“Was this as fresh as labelled?”) — this feedback loop is itself a core data asset, see § 7.4.

6. Feature Requirements (MVP)

Each feature below is specified with: what the customer sees, what it requires operationally, and the priority for the pilot build.

6.1 Expiry-Based Filtering

Element	Requirement	Priority
Filter options	Three filter states: “Use Today”, “Fresh Stock”, “Any” (default). No numeric date-range filter in v1 — three states are simpler to tag manually and simpler for a first-time user to understand than a date picker.	P0
Use Today pricing	Auto-discount band of 20–40%, set per category by ops (not arbitrarily by store), applied automatically when an item is tagged “Use Today.” Store owner cannot override the discount percentage in v1 — this avoids inconsistent, untrustworthy discounting (see § 1.2).	P0
Fresh Stock pricing	Standard catalog price. No premium up-charge in v1 — “slightly premium pricing” from the original concept is dropped because it penalises the default choice and will suppress conversion; revisit only if pilot data shows appetite for it.	P0
Tagging input	Manual entry by store owner via a 3-tap flow (product → quantity → expiry bucket) on the store partner app. No barcode scanning or OCR in MVP — added complexity for a workflow that has to work for a non-technical owner first.	P0

6.2 Freshness Meter (Smart Label)

Element	Requirement	Priority
Visual	Three-colour badge on product card and product detail: green (Fresh Stock), amber (expiring in 2–3 days, shown as “Soon”), red (Use Today).	P0
Underlying data	Directly derived from the store's expiry-bucket tag (§ 6.1) — no separate “freshness score” algorithm in MVP. A computed score implies a precision the manual tagging input can't actually support.	P0
Manufacturing date display	Shown on product detail page when the store has entered it; optional field, not required to list a product — making it mandatory would block onboarding for stores with no batch records.	P1

6.3 Local Store Discovery & Catalog

Element	Requirement	Priority
Delivery radius	Fixed radius per store (e.g. 2–3 km) configured by ops at onboarding; not dynamically calculated in MVP.	P0
Catalog	Store owner lists products from a shared Quicky master catalog (to standardise names/images/categories) rather than free-text entry, with manual price + stock + expiry-bucket per listing.	P0
Store profile	Name, rating (seeded manually pre-launch since no orders exist yet), distance, and a “Verified freshness partner” badge once a store completes onboarding training.	P1

6.4 Checkout & Order Routing

Element	Requirement	Priority
Single-store orders only	MVP does not support cart items from multiple stores in one order — multi-store basket splitting adds delivery-routing complexity the pilot doesn't need to solve yet.	P0
Delivery fee	Flat fee per order in pilot zone (e.g. ₹15–25), waived above a minimum basket value to push basket size up given the unit-economics risk flagged in § 8.	P0
Freshness feedback prompt	Single tap rating shown after delivery confirmation (§ 5, step 7).	P0

6.5 Store Partner App / Dashboard

Element	Requirement	Priority
Order inbox	Incoming orders with accept/reject and a packing checklist pulled from the order.	P0
Inventory tagging	The 3-tap expiry tagging flow from § 6.1, plus daily reminder notification to re-tag stock (“Mark today’s near-expiry items”) — this nudge is necessary because stale tagging breaks customer trust in the Freshness Meter immediately.	P0
Basic sales view	Orders fulfilled, near-expiry units sold vs. listed — simple counts, not a full analytics suite (that's a v2/v3 revenue stream, § 7.3).	P1

6.6 Features explicitly deferred (not in MVP)

- Warranty countdown / electronics category — see § 9.
- AI-driven “sell this before expiry” recommendation engine for store owners — needs a volume of tagging history that won't exist at pilot scale; a simple rule (“item untouched 2

days after Use Today tag → nudge owner to discount further or pull listing”) covers the same need without ML infrastructure.

- **Subscription tier** (free delivery / early access) — premature before there's proven repeat usage to monetise.
- **Premium store placement** — revenue feature, not a pilot need, and risks undermining the “transparency” positioning if introduced too early.

7. Revenue Model

Sequenced by what the pilot can realistically support — not everything launches at once.

7.1 Pilot-phase revenue (live from day one)

- **Commission on orders:** 10–15% per order, at the lower end of the original 10–20% range. Near-expiry items are already discounted, so a high commission on top compresses store margin further and undermines the “stores recover money” pitch that gets them to onboard.
- **Delivery fee:** Flat per-order fee, waived above a basket-size threshold (§ 6.4) to encourage larger, more economically viable orders.

7.2 Deferred to post-pilot (only once volume justifies it)

- **Subscription tier:** Free delivery + early access to Fresh Stock listings, once repeat order frequency data shows there's something worth subscribing for.
- **Premium store placement:** Paid visibility for store listings — sequenced after pilot to avoid undermining transparency positioning before trust is established.
- **Store analytics product:** Inventory insights and demand prediction sold to store partners, once enough stores and order history exist to make the insights non-trivial.

8. Unit Economics (Pilot Assumptions)

This is the single biggest open risk in the original concept and deserves explicit numbers rather than a one-line “challenge.” Quick-commerce as a category remains structurally low-margin to cash-negative at this stage of its evolution, which is exactly why this needs to be modelled before scaling spend, not after.

8.1 Basket size & delivery cost — Delhi pilot estimates

Line item	Pilot estimate (Delhi)	Source / assumption
Avg. basket size — Use Today order	₹120–180	Single near-expiry item (bread ₹35, dairy ₹60–100, snacks ₹45–60). Needs validation in pilot week 1.
Avg. basket size — Fresh Stock order	₹280–420	Mixed basket (2–3 grocery items). Healthier economics;

Line item	Pilot estimate (Delhi)	Source / assumption
		encourage with free-delivery threshold.
Target mixed basket (Use Today + Fresh Stock)	₹350+	Goal: at least one Use Today item bundled with regular stock per order — this is the unit-economics fix, not a coupon strategy.
Delivery partner cost / order (Delhi)	₹45–60	Typical gig-delivery rate for 2–3 km, 30-min SLA in Delhi NCR. Negotiate a pilot rate with a local aggregator (e.g. Dunzo, Porter, or a direct hire).
Commission revenue / order at ₹300 basket	₹30–45 (10–15%)	Covers delivery cost only at the high end of basket and low end of commission — hence the need for delivery fee on small baskets.
Delivery fee (small baskets <₹250)	₹25–30	Bridges the gap on small-basket orders. Waived above threshold to incentivise larger orders.
Break-even basket size (no delivery fee)	₹350–400	At 12% commission, this yields ₹42–48 commission, roughly covering delivery cost. Set free-delivery threshold at ₹349.

These are directional estimates based on Delhi NCR gig-delivery costs and typical kirana basket behaviour. Instrument actual per-order cost from day one of the pilot and run a weekly P&L review — if avg. basket stays below ₹200 after 3 weeks, the free-delivery threshold and category mix need revisiting before scaling headcount or store count.

8.2 “Use Today” discount band — category-level rationale

The original concept used 20–40% as a placeholder. Below is a category-level hypothesis based on typical kirana gross margins in Delhi, with the goal of leaving the store with a positive margin even on discounted near-expiry stock. These are starting points, not fixed numbers — revisit monthly using pilot sell-through data.

Category	Typical kirana gross margin	Proposed Use Today discount	Store margin after discount	Rationale
Bread / baked goods	18–22%	25%	Negative (loss leader)	Write-off anyway within 24h; 25% discount is better than

Category	Typical kirana gross margin	Proposed Use Today discount	Store margin after discount	Rationale
				zero recovery. Keep discount capped at 25% to limit pain.
Dairy (milk, curd, paneer)	12–18%	15–20%	Near zero to slight loss	Very short shelf life; customers have high price sensitivity. 15% for same-day expiry, 20% for use-today.
Packaged snacks / biscuits	20–28%	20%	0–8% margin retained	Longer shelf life means fewer truly distressed items; 20% discount moves stock without destroying margin.
Staples (atta, rice, dal)	8–12%	10%	Breakeven to slight loss	Rarely expire but sometimes batch-date issues. Small discount only; margin is already thin.
Beverages (juices, soft drinks)	20–30%	20–25%	0–10% margin retained	Expiry-sensitive in summer; 20–25% is competitive vs. local

Category	Typical kirana gross margin	Proposed Use Today discount	Store margin after discount	Rationale
				market sale pricing.

Key constraint: Quicky does not set the MRP — it applies a discount to the store's listed price. Stores should list at standard MRP, and the platform applies the Use Today discount band automatically. If a store manually lists at an inflated price to absorb the discount, the onboarding agreement and ops team need to flag and correct this.

9. Data & Tagging Workflow

This is the operational backbone the original concept under-specified. If this doesn't work for a real kirana owner, nothing else in this document matters.

9.1 Master catalog

Quicky maintains a shared product catalog (name, category, image, standard unit) so store owners select from a list rather than typing free text. This keeps listings consistent and searchable, and is the first thing to build before any store onboarding happens.

9.2 Store-side tagging — the 3-tap flow

1. Store owner picks a product from the catalog.
2. Enters quantity in stock.
3. Selects an expiry bucket: Use Today / Fresh Stock (no third bucket in v1 — reduces decision friction for the owner).
4. Optionally enters manufacturing date (skippable).

Design target: under 30 seconds per product. This is a hard UX constraint, not a stretch goal. A store owner with 8–10 near-expiry items to tag has a 4–5 minute tolerance at most before the task feels like a burden and gets skipped. Below is the pre-launch usability test protocol to validate this before any customer-facing code is written.

9.3 Tagging usability test protocol

Run this test before building the customer app. Results gate whether the tagging flow is launch-ready.

Test element	Specification
Participants	5–8 kirana store owners from the target pilot zone (Rohini / North Delhi). Recruit in person; do not use digital screeners — the whole point is to test with non-tech-native users.
Device	Android mid-range handset (e.g. Redmi 10C or similar ₹8,000–12,000 price range) — this is the device your actual store owners will use, not a tester's iPhone.

Test element	Specification
Task	Tag 5 products from a prepared list (mix of bread, dairy, packaged snack) as Use Today or Fresh Stock, entering quantity for each. No instructions beyond what the app shows.
Success threshold	Median time per product ≤ 30 seconds AND ≥ 4 of 5 participants complete all 5 tags without asking for help. Below this → redesign before launch.
Error tracking	Count: wrong bucket selected (Use Today vs Fresh Stock confused), quantity entry errors, drop-offs mid-flow. Any error on $>40\%$ of tasks = that specific step needs redesign.
Language	UI must be available in Hindi for this test. English-only will artificially inflate failure rates and waste test time.
Output	A one-page test report: median time, error rates by step, qualitative quotes from participants. Share with the whole team; this data should inform every subsequent iteration of the store-side app.

9.4 Staleness handling

- Daily push reminder to re-tag stock (§ 6.5).
- A listing not re-confirmed within 48 hours is auto-flagged “unverified” and de-prioritised in search results — protects customer trust in the Freshness Meter without requiring real-time POS sync.

9.5 Warranty data — explicitly out of MVP

No warranty fields, countdown, or electronics category in the pilot catalog (see § 3.2 and § 11.1 for the v2 rationale and conditions for revisiting).

10. Success Metrics (Pilot)

The pilot exists to answer four specific questions — metrics are organised around them, not around vanity totals.

Question	Metric	Pilot target (directional, refine after week 1)
Will customers choose near-expiry stock when it's clearly labelled?	% of orders containing at least one “Use Today” item	Track from week 1; no target until baseline is known
Does this measurably reduce store write-offs?	Store-reported near-expiry units sold via app vs. prior written-off volume (self-reported baseline at onboarding)	Directional reduction visible by week 4

Question	Metric	Pilot target (directional, refine after week 1)
Is the tagging workflow sustainable for a non-technical owner?	% of onboarded stores still actively tagging stock daily after 2 weeks	≥70% — below this, the workflow itself needs redesign before anything else
Are unit economics viable at small scale?	Commission + delivery fee revenue per order vs. delivery cost per order	Track weekly; this is the kill/continue gate for scaling beyond pilot, see § 8

11. Risks & Open Questions

Risk	Why it matters	Mitigation / next step
Manual expiry tagging doesn't stick with store owners	The entire product depends on this data being current; this is the single highest-risk dependency in the whole plan.	Design and usability-test the 3-tap flow (§9.2) with real kirana owners before writing a line of customer-facing code.
Near-expiry orders are too small to be economically viable	Could mean the pilot “works” on engagement but loses money on every order.	Instrument basket size and delivery cost from day 1 (§8); set a hard go/no-go threshold before scaling spend.
Customers don't think about expiry when ordering online	If “Use Today” adoption stays near zero, the differentiation doesn't exist in practice.	Pilot in a price-sensitive neighbourhood with Persona A (§4.1) as the lead segment, and message the discount explicitly rather than relying on the filter to be discovered.
Pricing-authority disputes with store owners	If owners feel the platform-set discount erodes their margin unfairly, onboarding and retention both suffer.	Be explicit at onboarding about the fixed discount band (§6.1) and revisit the band per category using pilot sell-through data.
Big platforms could copy the feature	Blinkit/Zepto/Instamart could add an expiry filter as a checkbox feature without changing their dark-store model.	The defensible part isn't the filter UI — it's the local-store relationship and the trust built around accurate tagging. Treat store partnerships as the moat, not the feature.

12. MVP Build Scope (Recap)

Concretely, for the pilot:

- 5–10 local stores, single neighbourhood (e.g. a defined zone within Rohini, Delhi, per the original concept).

- Grocery & daily essentials category only — no electronics, no warranty fields.
- Manual expiry tagging via the 3-tap flow (§9.2), two buckets only: Use Today / Fresh Stock.
- Fixed platform-set discount band for Use Today items (§6.1) — no store-side override.
- Single-store checkout, flat delivery fee with a free-delivery basket threshold (§6.4).
- Post-delivery freshness feedback prompt (§5, §6.4) — this is the cheapest, highest-value instrumentation in the whole MVP and should not be cut.

12.1 What this PRD deliberately does not include yet

Warranty tracking, AI recommendations, subscriptions, premium placement, multi-store carts, and real-time POS integration are not missing by oversight — they're sequenced for after the pilot proves the three things that actually carry risk: tagging sustainability, customer behaviour change, and unit economics. Build those, prove them, then expand scope.

13. Tech Stack

The tech stack below is specified for the MVP pilot only. It prioritises shipping speed and a single codebase over performance optimisation — the right call at this stage, where the risk is validating behaviour, not handling scale.

13.1 Mobile app — React Native

Both the customer app and the store partner app will be built in React Native. This means a single codebase targeting Android and iOS simultaneously, with no separate native builds to maintain during the pilot.

Decision	Rationale
Framework: React Native (not Flutter, not native)	React Native is the right call for this product at this stage. It has a larger Indian developer talent pool than Flutter, well-established libraries for the specific needs of a quick-commerce app (maps, payments, camera for barcode scanning if added in v2), and is battle-tested by apps like Myntra and Nykaa at production scale. The tradeoff vs. native performance is irrelevant at pilot scale.
Target platform priority: Android first	Your users (kirana store owners, price-sensitive urban consumers in Delhi) skew heavily Android. iOS can be in the binary from day one via the shared codebase, but QA priority and device testing in the pilot should be Android-first. Specifically: Redmi, Samsung M-series, Realme — not flagships.
Language: TypeScript (not plain JavaScript)	No negotiation on this. TypeScript adds compile-time safety that prevents an entire class of bugs that are expensive to debug in a fast-moving MVP codebase. Set up strict mode from day one.
Navigation: React Navigation v6+	The de facto standard; avoid Expo Router in v1 unless the team is already familiar with it, as the debugging surface is wider.

Decision	Rationale
State management: Zustand or React Context	Zustand for any state shared across screens (cart, user session, active filters); Context for simpler local-ish state. Avoid Redux at this scale — the boilerplate cost is not justified for a pilot.
Maps / location: react-native-maps + Google Maps SDK	Required for store proximity, delivery tracking, and radius configuration. Google Maps Platform pricing is negligible at pilot volume; don't over-engineer with alternatives at this stage.

13.2 Backend

Layer	Recommended choice	Note
API	Node.js + Express or Fastify	Fast to build, easy to hire for in India, sufficient for pilot load. Move to a more structured framework (NestJS) in v2 if codebase complexity grows.
Database	PostgreSQL	Relational data (orders, products, stores, expiry tags) fits a relational model cleanly. Use Supabase for managed hosting to reduce ops overhead during the pilot.
Auth	Firebase Auth or Supabase Auth	Phone number OTP is the standard auth flow for Indian consumer apps. Both support it out of the box; don't build auth from scratch.
Push notifications	Firebase Cloud Messaging (FCM)	Required for the daily store-owner re-tag reminder (§ 9.4) and customer order updates. FCM is free at pilot volume.
File storage (product images)	Cloudflare R2 or AWS S3	Product catalog images need a CDN-backed store. R2 has no egress fees; prefer it over S3 for cost at this scale.

13.3 Payments

Integrate Razorpay as the payments layer. It supports UPI, cards, net banking, and wallets — all the methods your users in Delhi will expect — and has a well-documented React Native SDK. Do not build payment handling yourself; Razorpay's settlement and refund workflows are a significant engineering saving.

Cash on delivery (COD) must also be supported in the pilot. A non-trivial share of kirana customers and first-time online buyers will not trust card/UPI on a new app. Gating COD until "v2" is a conversion mistake.

13.4 What to not build in MVP

- A custom CMS for the product catalog — a shared Google Sheet or Notion database maintained by ops is sufficient for 5–10 stores and a few hundred SKUs. Build the CMS only when the ops team is spending more than 2 hours/week on catalog maintenance.

- A real-time inventory sync with store POS systems — already deferred in § 3.2. Manual tagging via the app is the integration for now.
- A data analytics dashboard — export order data to a Google Sheet or Metabase instance. Don't write a custom analytics product until there's enough data to make it useful and a clear buyer for it inside the store-partner relationship.
- Any ML / recommendation system — deferred (§ 6.6). The pilot data doesn't exist yet to train or evaluate a model.